

UAV MOTION PLANNING HANDOUT

Dijkstra and A*

Detailed algorithms, derivations, proofs, and planning interpretation

Bijay Gaudel

Purpose. Given a weighted graph with nonnegative edge costs, Dijkstra computes least-cost paths by ordering vertices with g . A* adds a heuristic h and orders them with $f = g + h$, focusing the search while retaining optimality under precise conditions. In motion planning, vertices represent collision-free configurations and edges represent feasible local motions.

1 Problem statement

Let $G = (V, E)$ be a finite directed graph. Each edge $(u, v) \in E$ has cost

$$c(u, v) \geq 0.$$

For a path $\pi = (v_0 = s, v_1, \dots, v_k = v)$, define

$$J(\pi) = \sum_{i=0}^{k-1} c(v_i, v_{i+1}).$$

The true shortest-path distance from source s to v is

$$\delta(s, v) = \min_{\pi: s \rightsquigarrow v} J(\pi),$$

with $\delta(s, v) = \infty$ if v is unreachable. The algorithm must return $g(v) = \delta(s, v)$ and a predecessor map from which a shortest path can be reconstructed.

For a grid planner, one common edge cost is Euclidean travel length,

$$c(u, v) = \|p_v - p_u\|_2,$$

possibly augmented by risk, energy, or clearance penalties. Dijkstra remains valid only if the resulting total edge costs are nonnegative.

2 From the Bellman equation to relaxation

Every shortest path to $v \neq s$ ends by entering v from some predecessor u . Therefore

$$\delta(s, s) = 0, \quad \delta(s, v) = \min_{(u,v) \in E} [\delta(s, u) + c(u, v)].$$

We do not know δ initially, so maintain tentative upper bounds $g(v)$:

$$g(s) = 0, \quad g(v) = \infty \quad (v \neq s).$$

Whenever an edge (u, v) is examined, perform *relaxation*:

$$\text{if } g(u) + c(u, v) < g(v), \quad g(v) \leftarrow g(u) + c(u, v), \quad \text{parent}(v) \leftarrow u.$$

The parent assignment records the final edge of the currently best known path. Because every finite $g(v)$ is the cost of an actual discovered path, it is always an upper bound:

$$g(v) \geq \delta(s, v).$$

3 Why the smallest tentative distance is safe

Dijkstra's key idea is to repeatedly finalize the unsettled vertex with minimum g .

Invariant 1 (Settled-vertex invariant). *Immediately after a vertex u is removed from the priority queue with the smallest valid key, $g(u) = \delta(s, u)$. Its distance never needs to change again.*

The intuition is a wavefront. With nonnegative edge costs, any route that first travels farther than the current wavefront cannot later become cheaper by adding more edges. This is precisely what fails when negative edges are allowed.

4 Detailed algorithm

The practical version below uses a binary min-heap and permits duplicate heap entries. This avoids needing a decrease-key operation. A popped entry is discarded when its key is stale.

Algorithm 1 Dijkstra with early goal termination and lazy heap deletion

Require: Graph $G = (V, E)$, nonnegative cost c , source s , optional goal t

Ensure: Distances g and predecessor map parent

```

1: for all  $v \in V$  do
2:    $g(v) \leftarrow \infty$ ;  $\text{parent}(v) \leftarrow \perp$ 
3:  $g(s) \leftarrow 0$ 
4:  $Q \leftarrow$  min-heap containing  $(0, s)$ 
5:  $S \leftarrow \emptyset$  ▷ settled vertices
6: while  $Q$  is not empty do
7:    $(d, u) \leftarrow \text{popMin}(Q)$ 
8:   if  $d \neq g(u)$  then ▷ stale heap entry
9:     continue
10:  if  $u \in S$  then
11:    continue
12:   $S \leftarrow S \cup \{u\}$ 
13:  if  $u = t$  then
14:    break ▷ the goal is now optimal
15:  for all  $(u, v) \in E$  do
16:     $d_{\text{new}} \leftarrow g(u) + c(u, v)$ 
17:    if  $d_{\text{new}} < g(v)$  then
18:       $g(v) \leftarrow d_{\text{new}}$ 
19:       $\text{parent}(v) \leftarrow u$ 
20:       $\text{push}(Q, (g(v), v))$ 
21: return  $g, \text{parent}$ 

```

Path reconstruction. If $g(t) < \infty$, start at t and repeatedly follow parent until s is reached; reverse the resulting list. If $g(t) = \infty$ after the queue empties, no graph path from s to t exists.

5 Correctness proof

Lemma 1 (Upper-bound property). *At every iteration, each finite $g(v)$ equals the cost of some discovered s - v path. Hence $g(v) \geq \delta(s, v)$.*

Proof. Initially only $g(s) = 0$ is finite and corresponds to the empty path. A relaxation sets $g(v) = g(u) + c(u, v)$, which appends edge (u, v) to the discovered path represented by $g(u)$. Thus every finite label remains the cost of a real path. No real path can cost less than the minimum over all paths, so $g(v) \geq \delta(s, v)$. $\square$

Theorem 1 (Dijkstra correctness). *When a vertex u is settled, $g(u) = \delta(s, u)$. Consequently, when the algorithm terminates, every reachable settled vertex has its exact shortest-path distance; if it stops when goal t is popped, the reconstructed s - t path is optimal.*

Proof. Assume for contradiction that u is the first settled vertex for which $g(u) > \delta(s, u)$. Consider a true shortest path P from s to u . Since s is settled first, P contains a first unsettled vertex y at the moment u is selected. Let x be the predecessor of y on P ; x is settled.

Because u is the first incorrectly settled vertex, $g(x) = \delta(s, x)$. When x was settled, edge (x, y) was relaxed, so

$$g(y) \leq g(x) + c(x, y) = \delta(s, x) + c(x, y) = \delta(s, y).$$

The upper-bound lemma gives $g(y) \geq \delta(s, y)$, hence $g(y) = \delta(s, y)$. Since y lies on a shortest path to u and all remaining edge costs are nonnegative,

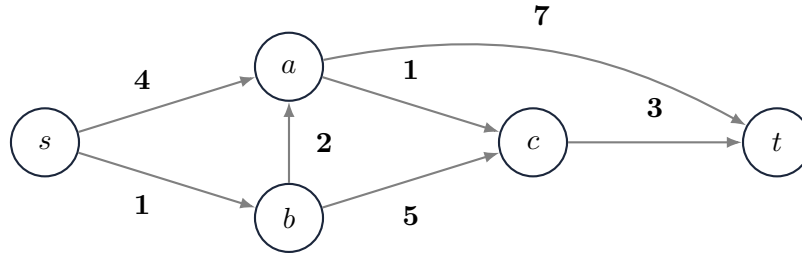
$$g(y) = \delta(s, y) \leq \delta(s, u) < g(u).$$

But y is unsettled and has a smaller queue key than u , contradicting the choice of u as the minimum. Therefore every settled label is exact. $\square$

Why nonnegative costs matter. Suppose $s \rightarrow a$ costs 2, $s \rightarrow b$ costs 5, and $b \rightarrow a$ costs -10 . Dijkstra settles a at cost 2, but the later route $s \rightarrow b \rightarrow a$ costs -5 . The settled-vertex invariant fails. Use Bellman–Ford or another method when negative edges are possible.

6 Worked example

Consider the graph below. Edge labels are costs.



Step	popped	$g(a)$	$g(b)$	$g(c)$	$g(t)$
0	—	∞	∞	∞	∞
1	s	4	1	∞	∞
2	b	3	1	6	∞
3	a	3	1	4	10
4	c	3	1	4	7
5	t	3	1	4	7

The predecessor chain is $t \leftarrow c \leftarrow a \leftarrow b \leftarrow s$, so the optimal path is

$$s \rightarrow b \rightarrow a \rightarrow c \rightarrow t, \quad J^* = 1 + 2 + 1 + 3 = 7.$$

Notice that a is first discovered with cost 4, then improved to 3 by relaxing (b, a) . A tentative label is not final until its vertex is popped with the minimum valid key.

7 Complexity and data structures

Let $|V|$ be the number of vertices and $|E|$ the number of directed edges.

Priority structure	Time	Typical use
Array / linear scan	$O(V ^2 + E)$	Dense or very small graphs
Binary heap	$O((V + E) \log V)$	Standard sparse planning graphs
Fibonacci heap	$O(E + V \log V)$ amortized	Mainly theoretical advantage

Memory is $O(|V| + |E|)$ when the graph is explicitly stored, plus $O(|V|)$ for distances, parents, and queue state. In an implicit grid, neighbors can be generated on demand, avoiding explicit storage of all edges.

For a d -dimensional grid with q cells per coordinate,

$$|V| = q^d.$$

This is the curse of dimensionality: even a polynomial-time graph algorithm can be expensive because the discretized graph itself grows exponentially with dimension.

8 Dijkstra as a motion planner

1. **Discretize the free configuration space.** A vertex may represent a 2-D position, a 3-D voxel, or a richer state such as position, velocity, and heading.
2. **Generate valid local motions.** Add an edge only if the local transition is collision-free and satisfies the chosen kinematic or dynamic constraints.
3. **Assign nonnegative costs.** Distance-only planning uses path length. Multi-objective planning may use

$$c(u, v) = w_\ell \ell(u, v) + w_E E(u, v) + w_R R(u, v) + w_T T(u, v), \quad w_i \geq 0.$$

Normalize terms carefully so one unit does not dominate merely because of scale.

4. **Run Dijkstra and reconstruct.** The result is optimal on this graph, not automatically in continuous space.
5. **Post-process if necessary.** A grid path is often jagged. Shortcutting, spline fitting, minimum-snap optimization, and time parameterization may be required before flight.

Resolution completeness. On a finite graph, Dijkstra finds a path if one exists. Relating that statement to the continuous planning problem requires assumptions: a valid continuous path should have positive obstacle clearance, and the grid resolution and motion primitives must be fine enough to represent it.

9 Dijkstra, uniform-cost search, and A*

Dijkstra is uniform-cost search on a graph. A* changes only the queue key:

$$\underbrace{g(n)}_{\text{cost from start}} \longrightarrow \underbrace{f(n) = g(n) + h(n)}_{\text{estimated solution cost}}.$$

Setting $h \equiv 0$ makes A* exactly Dijkstra. If h is admissible, $h(n) \leq h^*(n)$, A* remains optimal. A consistent heuristic additionally satisfies

$$h(n) \leq c(n, n') + h(n')$$

for every edge, which makes f nondecreasing along paths and avoids reopening closed vertices in the usual graph-search implementation. Dijkstra has no directional goal information; its wavefront expands in all low-cost directions. A good heuristic reduces expansions while preserving the optimality guarantee.

10 A* search in full detail

Let t be the goal. A* maintains the same tentative cost-to-come $g(n)$ and predecessor map as Dijkstra, but orders the open queue by

$$f(n) = g(n) + h(n),$$

where $h(n)$ estimates the remaining cost from n to t . The true remaining cost is

$$h^*(n) = \delta(n, t).$$

Thus $f(n)$ estimates the cost of a complete solution constrained to pass through n . The three values have distinct roles:

- $g(n)$ is computed from discovered edges and is the cost of a concrete start-to- n path;
- $h(n)$ is problem knowledge supplied by the designer;
- $f(n)$ is only a queue priority, not an independently accumulated path cost.

Algorithm 2 A* graph search with reopening and lazy heap deletion

Require: Graph $G = (V, E)$, nonnegative cost c , source s , goal t , heuristic h

Ensure: An optimal s - t path when h is admissible and required reopenings are allowed

```

1: for all  $v \in V$  do
2:    $g(v) \leftarrow \infty$ ;  $\text{parent}(v) \leftarrow \perp$ 
3:  $g(s) \leftarrow 0$ ;  $Q \leftarrow$  min-heap containing  $(h(s), s)$ 
4: while  $Q$  is not empty do
5:    $(f_{\text{old}}, u) \leftarrow \text{popMin}(Q)$ 
6:   if  $f_{\text{old}} \neq g(u) + h(u)$  then
7:     continue ▷ stale entry
8:   if  $u = t$  then
9:     return reconstruct( $\text{parent}, t$ )
10:  for all  $(u, v) \in E$  do
11:     $g_{\text{new}} \leftarrow g(u) + c(u, v)$ 
12:    if  $g_{\text{new}} < g(v)$  then
13:       $g(v) \leftarrow g_{\text{new}}$ ;  $\text{parent}(v) \leftarrow u$ 
14:      push( $Q, (g(v) + h(v), v)$ ) ▷ also reopens  $v$ 
15: return failure

```

The reopening behavior matters for admissible but inconsistent heuristics. With a consistent heuristic, a vertex need not be reopened after expansion, and a conventional closed set is safe.

11 Admissibility and consistency

- A heuristic is **admissible** when it never overestimates:

$$0 \leq h(n) \leq h^*(n) = \delta(n, t), \quad h(t) = 0.$$

- It is **consistent** (or monotone) when, for every edge (n, n') ,

$$h(n) \leq c(n, n') + h(n').$$

This is the triangle inequality for the heuristic estimate.

Lemma 2 (Consistency implies admissibility). *If $h(t) = 0$ and h is consistent, then $h(n) \leq h^*(n)$ for every vertex that can reach t .*

Proof. Take any path $n = v_0, v_1, \dots, v_k = t$. Repeated consistency gives

$$h(v_0) \leq c(v_0, v_1) + h(v_1) \leq \dots \leq \sum_{i=0}^{k-1} c(v_i, v_{i+1}) + h(t) = J(\pi).$$

Because this holds for every path from n to t , it holds for the cheapest one: $h(n) \leq h^*(n)$. $\square$

Lemma 3 (f -monotonicity). *If h is consistent, then along every relaxed tree edge (n, n') ,*

$$f(n') = g(n') + h(n') \geq g(n) + h(n) = f(n).$$

Proof. For a tree edge, $g(n') = g(n) + c(n, n')$. Consistency rearranges to $c(n, n') + h(n') \geq h(n)$. Adding $g(n)$ proves the claim. $\square$

Therefore A* wavefront levels do not decrease along a path when h is consistent. Once a vertex is removed with minimum f , its best g is final, exactly the property needed for a permanent closed set.

Admissible does not necessarily mean consistent. An admissible heuristic can drop too sharply across an edge. Such a heuristic can cause a previously expanded vertex to receive a better g later. Optimal A* must then reopen that vertex. Tree-search A* remains optimal with admissibility, while graph-search A* without reopening generally requires consistency.

12 A* optimality proof

Theorem 2 (Optimality with an admissible heuristic). *Assume nonnegative edge costs, an admissible heuristic, and an implementation that permits every necessary reopening. When A* removes goal t from OPEN, $g(t) = \delta(s, t)$.*

Proof. Let $C^* = \delta(s, t)$. Before an optimal solution is returned, consider an optimal path from s to t . There is some frontier vertex n on this path whose predecessor portion has been discovered with optimal cost, so $g(n) = \delta(s, n)$. By admissibility,

$$f(n) = g(n) + h(n) \leq \delta(s, n) + \delta(n, t) = C^*.$$

A* removes a minimum- f vertex. If it were to remove the goal with a suboptimal value $g(t) > C^*$, then, because $h(t) = 0$,

$$f(t) = g(t) > C^* \geq f(n),$$

contradicting minimum- f selection. Therefore a popped goal has $g(t) = C^*$. $\square$

Correct stopping rule. Do not stop when the goal is generated or inserted into OPEN. Its current g may later improve. Stop when the goal is popped as the valid minimum- f entry under the assumptions above.

13 How a heuristic changes the search

For any cost threshold C , A* prioritizes the set

$$\{n : g(n) + h(n) \leq C\}.$$

With $h = 0$, this is Dijkstra's cost ball centered at the start. With a geometric heuristic, it becomes an ellipse-like region concentrated around promising start-to-goal routes. The heuristic changes the *order and number of expansions*, not the edge costs or the optimal path definition.

Suppose h_1 and h_2 are both admissible and

$$h_1(n) \leq h_2(n) \leq h^*(n) \quad \text{for all } n.$$

Then h_2 **dominates** h_1 . Subject to standard tie-breaking conditions, A* with h_2 expands no more nodes with $f < C^*$ than A* with h_1 . More informed admissible heuristics generally save work.

The unattainable ideal $h = h^*$ makes every expanded vertex lie on an optimal-cost contour; however, computing h^* is the original shortest-path problem. Heuristic design seeks a cheap lower bound that is as close as practical to h^* .

14 Constructing motion-planning heuristics

Four- and eight-connected 2-D grids

For unit-cost four-neighbor motion from (x, y) to (x_t, y_t) , Manhattan distance is exact in empty space:

$$h_{\text{Man}} = |x - x_t| + |y - y_t|.$$

For eight-neighbor motion with axial cost 1 and diagonal cost $\sqrt{2}$, let $d_x = |x - x_t|$ and $d_y = |y - y_t|$. The octile heuristic is

$$h_{\text{oct}} = \max(d_x, d_y) + (\sqrt{2} - 1) \min(d_x, d_y).$$

Euclidean distance

$$h_{\text{Euc}} = \sqrt{d_x^2 + d_y^2}$$

is admissible whenever path cost is at least physical Euclidean length. Manhattan distance is *not* admissible on an eight-connected grid with diagonal cost $\sqrt{2}$, because it overestimates a diagonal move.

Three-dimensional position and vehicle limits

For path-length cost in free space,

$$h(p) = \|p - p_t\|_2$$

is a lower bound because obstacles cannot make the shortest route shorter than the straight line. For minimum-time planning with speed limit v_{max} ,

$$h(p) = \frac{\|p - p_t\|_2}{v_{\text{max}}}$$

is admissible. Stronger bounds can incorporate acceleration, heading, or Dubins/Reeds–Shepp geometry, provided they solve a relaxation of the true problem and never exceed its optimal remaining cost.

Relaxations, landmarks, and combinations

Removing constraints produces lower bounds: ignore obstacles, ignore dynamics, or project a high-dimensional state into position space and solve the easier problem. Precomputed landmark distances can yield triangle-inequality bounds. If $h_1, \dots, h_k$ are admissible, then

$$h(n) = \max_i h_i(n)$$

is admissible and dominates every component. A nonnegative weighted sum is not automatically admissible unless its total is separately proved to remain below h^* .

15 Worked A* comparison

Return to the graph in Section 6 with goal t . Suppose a geometric relaxation gives

$$h(s) = 5, \quad h(a) = 4, \quad h(b) = 5, \quad h(c) = 3, \quad h(t) = 0.$$

These values are admissible: the true remaining costs are $h^*(s) = 7$, $h^*(a) = 4$, $h^*(b) = 6$, $h^*(c) = 3$, and $h^*(t) = 0$. They are also consistent for every graph edge.

Pop	update	resulting best g	queue keys $f = g + h$	next
s	discover a, b	$g(a) = 4, g(b) = 1$	$f(a) = 8, f(b) = 6$	b
b	improve a , discover c	$g(a) = 3, g(c) = 6$	$f(a) = 7, f(c) = 9$	a
a	improve c , discover t	$g(c) = 4, g(t) = 10$	$f(c) = 7, f(t) = 10$	c
c	improve t	$g(t) = 7$	$f(t) = 7$	t
t	stop	$g(t) = 7$	—	—

On this tiny graph, A* and Dijkstra pop the same optimal-path vertices. The heuristic benefit becomes visible when many off-route branches have low g but large estimated remaining cost: Dijkstra expands them, while A* postpones them because their f is high.

16 Tie-breaking, reopenings, and practical details

Many vertices may share the same f . Tie-breaking does not change optimality under the standard assumptions, but it changes work and path appearance. Common policies are:

- larger g (equivalently smaller h) first, often pushing search toward the goal;
- smaller g first, producing a broader wavefront;
- a deterministic vertex ID, improving reproducibility.

Use a lexicographic heap key such as $(f, -g, \text{ID})$. Floating-point comparisons may require care, but an overly large tolerance can suppress real improvements and invalidate guarantees.

With duplicate heap entries, a robust stale-entry test compares the popped key against the vertex's current $g + h$. With an inconsistent heuristic, do not permanently reject a better route merely because the vertex was expanded before; update g , replace its parent, and put it back into OPEN.

17 Weighted A* and bounded suboptimality

Weighted A* uses

$$f_w(n) = g(n) + wh(n), \quad w > 1.$$

It behaves more greedily and often expands fewer vertices, but wh is generally inadmissible. Under common assumptions, if h is admissible, the returned solution satisfies the bound

$$J_{\text{returned}} \leq wC^*.$$

This trades solution quality for speed. Greedy best-first search is the extreme strategy of ordering only by h ; it has no general optimality guarantee. Anytime variants can begin with a large weight, quickly produce a feasible route, and reduce the weight while reusing search information to improve the bound.

18 A* complexity and what the bound means

In the worst case, A* remains exponential in solution depth and can expand most reachable vertices. With a binary heap, graph bookkeeping still costs approximately

$$O((|V| + |E|) \log |V|),$$

but the practical objective is to touch a much smaller subset of the graph. Memory is often the limiting resource because OPEN, distance labels, and parents can grow with the number of generated vertices.

The theoretical optimal-efficiency result for A* is nuanced: among algorithms using the same consistent heuristic and satisfying suitable “A*-like” conditions, no alternative can guarantee expanding fewer nodes with $f < C^*$. Tie behavior at $f = C^*$ and implementation details can still change the observed count.

19 A* implementation checklist and common errors

- Prove the heuristic is a lower bound for the *same units and objective* as g .
- Set $h(t) = 0$ and keep g as actual accumulated cost; never add h into g .
- Stop when the goal is popped, not when it is first generated.
- Use consistency if vertices are permanently closed; otherwise implement reopening.
- Check stale queue entries after every pop.
- Do not use Manhattan distance with unrestricted diagonals unless the cost model makes it a valid lower bound.
- Collision-check edges even when the heuristic ignores obstacles.
- State whether the result is optimal, w -suboptimal, or merely feasible.
- Report expansions, generated vertices, peak OPEN size, path cost, and runtime when comparing heuristics.

20 Dijkstra implementation checklist and common errors

- Do not mark a vertex final when it is first discovered; finalize it only when popped with the smallest valid key.

- Handle stale heap entries or implement a correct decrease-key operation.
- Stop at the goal only when the goal is popped, not when it is first inserted.
- Use ∞ initialization and guard against numeric overflow in fixed-width integer implementations.
- Reject negative edge costs. Zero-cost edges are allowed.
- Store predecessors only when a strict improvement occurs; define a tie policy if deterministic paths matter.
- Collision-check every motion primitive, not only its endpoint.
- State the guarantee precisely: shortest path on the constructed graph under the selected edge-cost model.

21 Summary

Dijkstra follows from one safe greedy decision:

With nonnegative costs, the unsettled vertex of minimum g can be finalized.

Relaxation propagates upper bounds, the priority queue selects the next provably optimal vertex, and parent pointers recover the path. A* retains relaxation but changes the priority from g to

$$f(n) = g(n) + h(n).$$

Admissibility is the lower-bound condition used in the optimality proof; consistency is the stronger edge-wise triangle inequality that makes f monotone and allows permanent closing without reopenings. Dijkstra is exactly A* with $h = 0$. Both are complete and optimal on a finite graph under their stated assumptions, but practical performance depends on graph size, resolution, and—for A*—the quality and cost of the heuristic.